Built-In File Editor Research Report

What Users Love, Hate, and Expect from Lightweight Text Editors

Prepared for: SinglePane File Manager Project

Date: March 9, 2026

Scope: Typora, iA Writer, Obsidian, and comparable editors

Executive Summary

This report synthesizes user sentiment across three popular lightweight editors (Typora, iA Writer, Obsidian) and draws additional insights from CotEditor, BBEdit, Sublime Text, VS Code, and file manager built-in editors (Total Commander, Path Finder, Midnight Commander). The goal: define what a competent built-in editor in a macOS file manager must do, what it should do, and what would be over-engineering.

The core finding: Users of lightweight editors overwhelmingly value speed, simplicity, and "just works" behavior. The single most consistent complaint across all editors is when basic operations feel slow or require unnecessary steps. For a file manager's built-in editor, the bar is: open any text file instantly, edit it competently, and get out of the way. The editor should feel like a sharp knife, not a Swiss Army knife.

1. App-by-App Analysis

1.1 Typora - What Users Love and Hate

Consistently Praised:

Live WYSIWYG rendering. Typora's signature feature hides Markdown syntax and shows the rendered result inline. Users describe this as "What You See Is What You Mean" -- it reduces cognitive load by eliminating the split between source and preview.

Distraction-free interface. Clean, minimal UI with no sidebars or unnecessary chrome. Focus Mode dims everything except the current line. Typewriter Mode keeps the active line centered vertically.

Instant responsiveness. The app feels fast. No perceptible lag between typing and rendering.

Export versatility. PDF, HTML, Word, EPUB, LaTeX, and more. Users praise the breadth and quality of export options.

Syntax highlighting for code blocks. Supports approximately 100 programming languages within fenced code blocks.

Table editing. Visual table editor that makes Markdown tables bearable.

Keyboard shortcuts. Rich shortcut set that keeps hands on the keyboard.

Low price. $15 for 3 devices is frequently cited as fair and easy to justify.

Consistently Criticized:

No split view. Users want to see raw Markdown source and rendered output side-by-side. Typora only offers one view at a time (rendered or source).

Limited find and replace. Basic regex support exists, but users have requested fuzzy search, AND/OR operators, and the ability to search while ignoring Markdown markup in WYSIWYG mode. The find panel is functional but not powerful.

No iPad/mobile app. Deal-breaker for mobile writers.

Paid with slow updates. Since going paid, users feel feature development has slowed relative to the price.

Limited customization. Beyond CSS themes, there is not much the user can configure about the editing experience.

No collaboration features. Single-user only.

Key Takeaway for SinglePane: Typora proves that a clean, fast editing experience with good syntax highlighting and minimal UI overhead creates loyal users. The lesson: do not add features at the cost of responsiveness.

1.2 iA Writer - What Users Love and Hate

Consistently Praised:

- **Focus Mode.** Dims everything except the current sentence (or paragraph). When combined with full-screen dark mode, users describe it as the most focused writing experience available. Foreign language teachers particularly love it.
- **Syntax Highlighting for prose.** Unique feature: highlights parts of speech (adjectives in brown, nouns in red, adverbs in purple, verbs in blue, conjunctions in green). This helps writers analyze sentence structure at a glance.
- **Speed and reliability.** App launches instantly, syncs flawlessly via iCloud and Dropbox, and never crashes.
- **Content Blocks.** Include images, CSV tables, text files, and code blocks as embedded content within documents.
- **Design quality.** Described as "the gold standard for focused markdown writing" -- the app that defined the category. The typography and spacing feel intentional and refined.
- **Cross-device sync.** iCloud sync works perfectly between Mac, iPhone, and iPad.

Consistently Criticized:

- **No true WYSIWYG.** Unlike Typora, iA Writer does not render Markdown inline. Users see the raw syntax while typing, which some find distracting.
- **No library organizer.** Users want a tree-styled library layout for organizing files into projects, chapters, and categories. The current flat organization is inadequate for long-form work.
- **Limited to short-form writing.** Users report it works well for blogs, notes, and short stories, but lacks the organizational features needed for books or complex research.
- **No index card view.** Writers want something like Scrivener's Cork Board for rearranging sections.
- **Limited customization.** The deliberate minimalism means users cannot change much about the editing experience.

Key Takeaway for SinglePane: iA Writer's Focus Mode concept is interesting but irrelevant for a file manager editor. The real lesson is that iA Writer's speed, reliability, and "it just works" quality create trust. Users tolerate missing features when the core experience is rock-solid.

1.3 Obsidian - What Users Love and Hate

Consistently Praised:

- **Local-first file storage.** All data lives as plain Markdown files on disk. Users have full control and ownership. No vendor lock-in.
- **Plugin ecosystem.** 1,000+ community plugins extend the editor in every imaginable direction. This is both a strength and a weakness (see below).
- **Bidirectional linking and graph view.** Creating a network of interconnected notes is Obsidian's core value proposition.
- **Live Preview mode.** Added after years of user requests (inspired by Typora), this renders Markdown inline while

editing. Users praised this addition extensively.

Performance with large vaults. Desktop version handles 50,000+ files with 40+ active plugins smoothly. Mobile has improved significantly.

Standard Markdown. Users appreciate that Obsidian uses standard Markdown without proprietary extensions, making files portable.

Auto-pairing. Built-in auto-pairing for brackets, quotes, and Markdown syntax (backticks, asterisks). Configurable in settings. Users can also install the Easy Typing plugin for enhanced auto-pairing behavior.

Consistently Criticized:

No native regex find and replace. This is a major, long-standing complaint. Users must install third-party plugins (Regex Find and Replace, Live Regex Find/Replace) for regex search within files. The built-in find/replace is basic.

Steep learning curve. Especially for users unfamiliar with Markdown. The sheer number of plugins and configuration options creates decision paralysis.

Plugin maintenance burden. Users describe "constant tinkering with plugin updates, compatibility checks, and dealing with broken plugins" as exhausting. What starts as flexibility becomes a chore.

Search limitations. Native search lacks Boolean operators, custom queries, and refined filters. Users find it inadequate for locating specific information in large vaults.

Performance with very large vaults. While desktop handles 50K files, users with hundreds of thousands of notes report slowdowns in file opening, searching, and navigation.

No native collaboration. Single-user only, problematic for team settings.

Syncing issues. Without Obsidian Sync (paid), cross-device syncing via iCloud or Dropbox can have conflicts and reliability issues.

Files must be in the vault. You cannot open an arbitrary file from anywhere on disk. The file must be located within your notes directory (the "vault").

Key Takeaway for SinglePane: Obsidian's biggest lesson is that missing native regex find/replace is a recurring pain point. For a file manager editor that will be used to edit config files, this is table-stakes. Also: do not require files to be in a specific location. A file manager editor must open any file, anywhere.

---

2. Find and Replace UX - Deep Analysis

Find and replace is one of the most-discussed editing features across all the editors studied. Here is what users expect and what frustrates them.

2.1 What Users Love

Instant match highlighting. As you type in the search field, all matches in the document should light up immediately. VS Code, Sublime Text, and Notepad++ all do this well. Users consider it essential.

Match count display. Showing "3 of 47 matches" in the search bar gives users confidence and orientation. VS Code and Typora both display this.

Regex support. Power users expect it. Obsidian's lack of native regex find/replace is one of its most-complained-about gaps. A toggle button (the .* icon) to enable regex mode is the standard pattern.

Case sensitivity toggle. A clickable Aa button in the search bar. Standard across VS Code, Sublime, Notepad++.

Whole word matching toggle. The \b or [ab] button. Less universally needed but expected by developers.

Preserve case on replace. VS Code added this and users praised it highly. When replacing "myVar" with "newVar", it preserves the casing of the original (e.g., "MyVar" becomes "NewVar", "MYVAR" becomes "NEWVAR"). This is a delight feature.

Search history. Being able to recall recent searches (last 10-16) via a dropdown or keyboard shortcut. EditPad Lite offers this and users value it for repetitive tasks.

Non-intrusive feedback. When no more matches exist, the best editors do not show a modal popup. Instead, the cursor stays put and the search button flashes briefly (EditPad Lite) or the match count shows "0 results" (VS Code). Modal popups interrupt flow.

Minimap integration. VS Code highlights search matches in the minimap scrollbar, giving a bird's-eye view of match distribution across the file.

2.2 What Users Hate

Modal dialog boxes for find/replace. The old TextEdit/Word model where a floating dialog covers the content. Users universally prefer inline panels.

Hidden search options. Xcode's original inline find panel hid regex, case, and word-match options behind a popup menu. Users could not see or discover them. Options should be visible toggle buttons, not buried in menus.

No incremental search. Editors that wait until you press Enter to show results feel broken to modern users.

Replace All without preview. Users want to see what will change before committing. VS Code's approach of highlighting replacements in context is praised.

Losing cursor position. If closing the find panel puts the cursor somewhere unexpected, users get disoriented.

Popup interruptions for "no results." A modal dialog saying "No matches found" that requires clicking OK is universally hated.

2.3 Recommended Design Pattern for SinglePane

Based on the research, the optimal find/replace panel for a file manager built-in editor:

1. Inline panel that slides in from the top of the editor (VS Code / Sublime model).
2. Search field with visible toggle buttons: Regex (.*), Case Sensitive (Aa), Whole Word ([ab]).
3. Instant incremental highlighting of all matches as the user types.
4. Match counter showing current position and total (e.g., "3 of 47").
5. Replace field that appears when toggled (Cmd+H), with Replace and Replace All buttons.
6. Keyboard-driven: Cmd+F opens find, Cmd+H opens find/replace, Enter moves to next match, Shift+Enter moves to previous, Cmd+G / Cmd+Shift+G for navigation.
7. Escape to close the panel and return focus to the editor at the current match position.
8. No modal dialogs for zero results -- just show "0 results" in the counter.
9. Search history accessible via up/down arrows in the search field.

---

3. Text Editing Conveniences

3.1 Auto-Pairing (Brackets, Quotes)

User expectations:

- Typing (should auto-insert) and place the cursor between them.
- Typing [should auto-insert]. Same for {/}, "/", '/', and backticks.
- Selecting text and typing (should wrap the selection in parentheses, not replace it.
- Pressing Backspace on an empty pair (e.g., cursor between ()) should delete both characters.
- Tab should jump the cursor past the closing character (escape the pair).
- This should be configurable -- some users (especially those editing prose) find auto-pairing annoying.

What frustrates users:

- Auto-pairing that cannot be turned off.
- Inconsistent behavior between character types (e.g., brackets auto-pair but quotes don't).
- Obsidian had bugs where the toggle to disable auto-pairing was broken, frustrating users who wanted it off.
- Auto-pairing backticks in non-Markdown contexts where it is unwanted.

Recommendation: Enable auto-pairing by default for code/config file types. Disable by default for plain text. Make it a per-file-type or global toggle.

3.2 Soft Wrap vs. Hard Wrap

The distinction:

Soft wrap (word wrap): Lines visually wrap at the window edge but no newline characters are inserted into the file. The file's content is unchanged.

Hard wrap: The editor inserts actual newline characters at a specified column width (typically 72 or 80 characters).

User expectations:

- Soft wrap should be the default for most file types. Users expect text to be visible without horizontal scrolling.
- Hard wrap should never happen automatically without explicit user action. Accidentally inserting newlines into config files, JSON, or code is a source of bugs.
- The wrap mode should be indicated in the status bar (or somewhere visible).
- A keyboard shortcut to toggle wrap on/off is expected (VS Code: Alt+Z).
- For config files (JSON, YAML, TOML), soft wrap is critical because these files often have long lines.

Recommendation: Default to soft wrap for all file types. Never auto-hard-wrap. Optionally expose a "Rewrap paragraph" command for prose editing, but this should be explicit, not automatic.

3.3 Tab Handling (Tabs vs. Spaces)

User expectations:

- Auto-detect the indentation style of the opened file and match it. VS Code does this well: it analyzes the file on open and adapts.
- Display the detected style in the status bar (e.g., "Spaces: 4" or "Tab Size: 4").
- Allow the user to click the status bar indicator to change the setting for the current file.
- Support EditorConfig files (.editorconfig) for project-level settings.
- Tab key should insert the configured indentation, not a literal tab character when spaces mode is active.
- Shift+Tab should outdent the current line or selection.

Recommendation: Auto-detect indentation on file open. Default to spaces (4) for new files. Show the setting in the status bar. Support .editorconfig.

3.4 Whitespace Visualization

User expectations:

- Toggle to show/hide invisible characters (spaces as dots, tabs as arrows, line endings as symbols).
- This is essential for config files, especially YAML where whitespace is syntactically significant.
- Should be easy to toggle on/off via a keyboard shortcut or menu item.
- Showing trailing whitespace (highlighted in a different color) is a bonus that developers appreciate.

Recommendation: Off by default. Toggleable via a keyboard shortcut. When enabled, show spaces as centered dots, tabs as arrows, and optionally highlight trailing whitespace.

---

4. Config File Editing (JSON, YAML, TOML, .env)

4.1 JSON Pain Points

No comments allowed. Users frequently want to annotate config files but JSON forbids comments. The editor should support JSONC (JSON with Comments) as VS Code does.

No trailing commas. Adding items to the end of arrays/objects creates noisy diffs. Users wish editors would be lenient about trailing commas.

Mandatory quoting. Every key must be quoted, which adds visual noise.

Bracket matching is critical. In deeply nested JSON, losing track of which } closes which { is the primary frustration. The editor must highlight matching brackets and indicate the nesting level.

Syntax validation. Immediate visual feedback when JSON is malformed (red squiggly underline or gutter icon) is expected.

4.2 YAML Pain Points

Whitespace sensitivity. The number-one complaint. Get indentation wrong and the file won't parse, often with cryptic error messages. The editor should:

- Show indentation guides (vertical lines at each indent level).
- Visualize spaces clearly (whitespace visualization toggle).
- Provide clear error highlighting for indentation mistakes.

Multiple representations for the same data. YAML allows strings without quotes, with single quotes, with double quotes, with block scalars, etc. This creates ambiguity and confusion.

Type coercion surprises. yes, no, on, off are interpreted as booleans. 1.0 might be a float. Users get bitten by this regularly.

4.3 TOML Pain Points

Less familiar syntax. Fewer users know TOML compared to JSON/YAML, so the editor should provide good syntax highlighting to aid comprehension.

Verbose for deep nesting. Table headers must be repeated at each nesting level.

Tooling gap. Syntax highlighting and validation support for TOML is less widespread than for JSON or YAML across editors.

4.4 .env Files

Simple format, simple needs. KEY=VALUE pairs, one per line. Comments with #.

Syntax highlighting for keys vs. values. Distinguish the variable name from its value visually.

No trailing whitespace insertion. Accidental whitespace in values can cause subtle bugs.

Quote handling. Values may or may not be quoted. The editor should not modify quoting behavior on save.

4.5 Recommendations for Config File Editing

1. Syntax highlighting for JSON, JSONC, YAML, TOML, .env, INI, XML, and shell scripts at minimum.
2. Bracket/brace matching with visual highlighting of the matching pair and the scope between them.
3. Indentation guides (vertical lines at each indent level) -- essential for YAML.
4. Basic syntax validation for JSON (malformed structure detection). Full schema validation is IDE territory and out of scope.
5. Folding for nested structures (collapse/expand objects, arrays, YAML blocks).
6. Auto-indent on Enter: pressing Enter should place the cursor at the correct indentation level based on context.

---

5. File Handling

5.1 Encoding Detection

User expectations:

- Auto-detect file encoding on open (UTF-8, UTF-16, Latin-1, Shift-JIS, etc.).
- Display the detected encoding in the status bar.
- Allow the user to change the encoding (re-interpret the file or convert on save).
- Default to UTF-8 for new files.
- Handle BOM (Byte Order Mark) correctly -- detect it, display its presence, and optionally strip it on save.

What CotEditor does well: Developed in Japan, CotEditor handles automatic encoding detection exceptionally well, including mixed-script text and vertical script. It also lists characters that cannot be converted when changing encoding, preventing data loss.

What BBEdit does well: Shows encoding in the status bar (off by default to reduce clutter, but easily enabled). Handles encoding changes gracefully.

Recommendation: Auto-detect encoding. Show it in the status bar. Default to UTF-8. Support converting between encodings on save with a warning about potential character loss.

5.2 Line Ending Handling (LF vs. CRLF)

User expectations:

- Auto-detect line endings on file open.
- Display the line ending type in the status bar (LF, CRLF, CR, or Mixed).
- Allow conversion between line ending types via a status bar click or menu option.
- Default to LF for new files on macOS.
- Detect and warn about mixed line endings (a common source of bugs).

Why this matters for a file manager editor: Users will open files that originated on Windows (CRLF), Linux (LF), or old Mac (CR). A file manager editor will encounter a wider variety of file origins than a typical writing app. Handling this correctly is table-stakes.

Recommendation: Auto-detect line endings. Show in status bar. Default to LF. Warn about mixed line endings.

5.3 Large File Support

The landscape:

- Most lightweight editors start to struggle at 10-50MB.
- VS Code loads entire files into memory, consuming 5GB of heap for a 5GB file.

- UltraEdit uses disk-based editing, loading only visible portions into memory.
- EmEditor uses SIMD (AVX-512) for fast large file handling.
- CotEditor was praised for opening hundreds-of-megabyte files in 1-3 seconds.

User expectations for a file manager editor:

- Files under 10MB should open instantly with full editing capability.
- Files 10MB-100MB should open quickly with possible feature degradation (disable syntax highlighting, minimap).
- Files over 100MB should open in a read-only or view-only mode, using memory-mapped I/O to display only the visible portion.
- The editor should never hang or crash on a large file -- it should gracefully degrade.

Recommendation: Use mmap(2) for large file viewing (consistent with the project's architecture). For files over a configurable threshold (e.g., 10MB), disable syntax highlighting and other expensive features. For files over 100MB, default to read-only view mode. Never load the entire file into memory.

---

6. The Line Between Lightweight Editor and IDE

6.1 Where the Boundary Exists

The distinction between a text editor and an IDE is defined by project-level features:

Feature	Lightweight Editor	IDE
Syntax highlighting	Yes	Yes
Find and replace (with regex)	Yes	Yes
Multiple file editing (tabs)	Yes	Yes
Auto-indent	Yes	Yes
Bracket matching	Yes	Yes
Code completion / IntelliSense	No	Yes
Compilation / Build	No	Yes
Debugging	No	Yes
Project-wide refactoring	No	Yes
LSP (Language Server Protocol)	No	Yes
Integrated version control	No	Yes
Test runner	No	Yes

The "lightweight IDE" category (VS Code, Sublime Text) blurs this line with plugins, but the core distinction remains: a text editor operates on individual files, while an IDE understands projects.

6.2 What SinglePane's Editor Should NOT Do

Based on this research, the following features would be over-engineering for a file manager's built-in editor:

Code completion / IntelliSense. This requires language servers, which are complex to implement and maintain.

LSP integration. Out of scope. This is IDE territory.

Integrated debugging. Not relevant to a file manager.

Project-wide search and replace. The file manager itself handles multi-file operations.

Git integration within the editor. The file manager has a separate Git module.

Snippet management. Nice to have in a dedicated editor, but not expected in a file manager's built-in editor.

Extension/plugin system for the editor. Adds complexity without proportional value.

Split editor panes (editing two files side-by-side within the editor). The file manager's dual-pane architecture serves this need at the file level.

Integrated terminal within the editor. The file manager already has a terminal module.

6.3 What SinglePane's Editor MUST Do

Based on consistent user expectations across all editors studied:

Absolute Must-Haves (table-stakes):

1. Syntax highlighting for 30+ common languages/formats
2. Line numbers
3. Find and replace with regex support
4. Undo/redo (unlimited)
5. Auto-indent (match current indentation on Enter)
6. Bracket/brace matching with highlighting
7. Encoding detection and display
8. Line ending detection and display
9. Soft word wrap (toggleable)
10. Tab/spaces auto-detection and configuration
11. File type detection (by extension and content)
12. Fast open (any file under 10MB in under 100ms)
13. Standard keyboard shortcuts (Cmd+S save, Cmd+Z undo, Cmd+F find, etc.)
14. Go to line number (Cmd+G or Ctrl+G)

Should-Haves (expected by power users):

1. Multiple cursors (Cmd+D to select next occurrence)
2. Code folding for nested structures
3. Indentation guides
4. Whitespace visualization toggle
5. Auto-pairing of brackets and quotes (configurable)
6. Current line highlighting
7. Minimap (scrollbar overview)
8. Configurable font and font size
9. Dark/light theme matching system appearance
10. Status bar showing: line/column, encoding, line endings, file type, indentation style
11. Selection count (characters, words, lines selected)
12. Drag and drop text
13. Column/block selection (Option+click+drag)

Nice-to-Haves (delight features):

1. Preserve case on replace (VS Code feature)

2. Search history in find panel
3. Color preview for hex/RGB color values in CSS/config files
4. Highlight trailing whitespace
5. Auto-close HTML/XML tags
6. Rainbow brackets (different colors for nesting levels)
7. Sticky scroll (show parent scope headers when scrolled deep into nested content)
8. Compare/diff two files (may be handled by the file manager itself)

---

7. Lessons from File Manager Built-In Editors

7.1 Path Finder

Path Finder's built-in text editor is described as "fine for simple editing and viewing" but inadequate for real work. It opens quickly and handles basic text files, but lacks syntax highlighting, find/replace, and other features expected by developers. Users tolerate it for quick peeks at file contents but switch to external editors for actual editing.

Lesson: If the built-in editor is too basic, users will ignore it entirely. It needs to be good enough that users don't reach for an external editor for 80% of quick-edit tasks.

7.2 Midnight Commander (mcedit)

Midnight Commander's built-in editor is surprisingly capable: syntax highlighting, regex find/replace, block operations, macro commands, auto-indent, and multiple-file editing. It handles files up to 64MB. This is the closest precedent to what SinglePane should build.

Lesson: A file manager's built-in editor can be much more capable than most people expect. mcedit proves that a well-implemented built-in editor becomes a genuine productivity tool, not just a viewer with an edit mode.

7.3 Total Commander

Total Commander includes a built-in viewer (F3) and relies on the user's configured external editor for editing (F4). The viewer is excellent (text, hex, image, multimedia), but editing is delegated.

Lesson: Total Commander's approach of excellent viewing + external editing works, but it introduces friction. SinglePane can differentiate by offering a capable built-in editor that eliminates the round-trip to an external app.

---

8. Synthesis: Priority Matrix

Tier 1 - Ship Without These and the Editor Is Useless

Feature	Rationale
Syntax highlighting (30+ languages)	Every editor since 2005 has this
Line numbers	Universal expectation
Find/replace with regex	Obsidian's biggest complaint is lacking this
Undo/redo (unlimited)	Fundamental editing operation
Auto-indent	Prevents frustration when editing nested files
Bracket matching	Critical for JSON, YAML, code files
Encoding detection + status bar display	Files come from everywhere; must handle gracefully
Line ending detection + status bar display	Cross-platform files are reality
Soft word wrap	Must not require horizontal scrolling for long lines
Fast open (under 100ms for files under 10MB)	This is a file manager -- speed is everything
Standard keyboard shortcuts	Cmd+S, Cmd+Z, Cmd+F, Cmd+G

Tier 2 - Ship Without These and Power Users Will Complain

Feature	Rationale
Multiple cursors	"Can't live without it" for Sublime/VS Code users
Code folding	Essential for navigating large config files
Indentation guides	Critical for YAML editing
Whitespace visualization	Debugging YAML/Python indentation
Auto-pairing (configurable)	Expected convenience
Current line highlighting	Visual orientation aid
Status bar (line/col, encoding, EOL, filetype, indent)	Standard information display
Tab/spaces auto-detection	Files should "just work" with correct indentation
Go to line number	Standard navigation feature
Column/block selection	Expected for tabular data editing

Tier 3 - Nice to Have, Builds Loyalty

Feature	Rationale

Feature	Rationale
Minimap	Helpful for long files, optional for short ones
Preserve case on replace	VS Code delight feature
Search history	Power user time saver
Color preview for hex values	Small delight when editing CSS/themes
Trailing whitespace highlighting	Developer hygiene feature
Rainbow brackets	Aids comprehension of deeply nested structures
Sticky scroll	Emerging pattern, not yet expected
Auto-close HTML/XML tags	Useful but narrow use case

Tier 4 - Over-Engineering (Do Not Build)

Feature	Rationale
Code completion / IntelliSense	IDE feature, not editor feature
LSP integration	Massive complexity, IDE territory
Debugging	Completely out of scope
Project-wide refactoring	File manager handles multi-file ops
Snippet system	Niche, better served by external tools
Plugin/extension system for the editor	Adds maintenance burden without proportional value
Split editor panes	Dual-pane file manager serves this need
Built-in terminal in editor	File manager already has terminal module
Markdown preview rendering	Not the purpose of this editor
AI code completion	IDE feature requiring cloud services

9. Implementation Recommendations

9.1 Syntax Highlighting Engine

Based on the research, two viable approaches exist for a native macOS editor:

Tree-sitter: Modern, creates full syntax trees, enables context-aware highlighting. Used by Zed, Neovim, and others. C library with Swift bindings available. Better accuracy but more complex to integrate.

TextMate grammars: Regex-based, enormous library of existing grammars covering hundreds of languages. Used by VS Code (via oniguruma). Simpler to implement but less context-aware.

Recommendation: Start with TextMate grammars for breadth of language coverage with lower implementation effort. Consider Tree-sitter for a future upgrade if deeper language understanding is needed (e.g., for bracket matching in context).

9.2 Status Bar Design

Based on CotEditor, BBEdit, and VS Code conventions, the status bar should display (left to right):

```
Ln 42, Col 17  |  UTF-8  |  LF  |  Spaces: 4  |  JSON  |  3 selected
```

Each element should be clickable to change its value (e.g., click "UTF-8" to change encoding, click "LF" to change line endings, click "Spaces: 4" to switch to tabs).

9.3 Performance Targets

Consistent with the SinglePane project's quality gates:

Operation	Target
Open file (< 1MB)	< 50ms to first paint
Open file (1-10MB)	< 200ms to first paint
Open file (10-100MB)	< 1s, degraded features
Open file (> 100MB)	View-only mode, mmap-based
Keystroke to character render	< 8ms (matching terminal target)
Find first result (incremental)	< 50ms
Syntax highlighting (full file < 1MB)	< 100ms

9.4 File Type Detection Priority

Based on the most common files users edit in a file manager context, prioritize syntax highlighting for these formats (in

order of importance):

1. JSON / JSONC
2. YAML
3. Shell scripts (bash, zsh, sh)
4. Markdown
5. TOML
6. .env / INI / Properties
7. XML / HTML
8. JavaScript / TypeScript
9. Python
10. Swift
11. CSS / SCSS
12. SQL
13. Dockerfile
14. Makefile
15. Ruby
16. Go
17. Rust
18. Java
19. C / C++ / Objective-C
20. PHP

---

10. Key Insights Summary

1. Speed is the feature. Across Typora, iA Writer, and CotEditor, the most consistently praised quality is responsiveness. Users forgive missing features but not sluggishness.
2. Find/replace with regex is table-stakes. Obsidian's biggest editing complaint is the lack of native regex find/replace. Do not ship without this.
3. The status bar is the editor's dashboard. Encoding, line endings, indentation style, cursor position, and file type should all be visible and clickable in the status bar. CotEditor and VS Code set the standard.
4. Auto-detect everything. Encoding, line endings, indentation style, and file type should be detected on open. Users should not need to configure these manually for each file.
5. Graceful degradation for large files. Use mmap, disable expensive features progressively, and never crash. CotEditor's ability to open 200MB files in seconds is praised specifically because most editors fail at this.
6. The inline find panel with visible toggles is the correct pattern. Not a floating dialog. Not hidden options. Visible toggle buttons for regex, case, and whole-word matching in an inline panel.
7. Multiple cursors have crossed from "power feature" to "expected feature." Users who have experienced them in Sublime/VS Code consider them essential.
8. The 80% rule: Build an editor good enough that users don't need to open an external editor for 80% of quick-edit tasks. The remaining 20% (complex refactoring, debugging, project-wide operations) should open in the user's preferred external editor via a configurable F4-style shortcut.

Sources

- Top 10 Markdown Editors in 2025 - DevOps School
- Typora - A Brief Review - BlakeSite
- Typora Reviews - Product Hunt
- Typora Reviews - G2
- Typora Find & Replace Enhancement - GitHub
- iA Writer Reviews - G2
- iA Writer Review 2026 - Elephas
- iA Writer Focus Mode - iA
- iA Writer Syntax Highlight - iA
- 2025 Obsidian Report Card - Practical PKM
- Obsidian is (almost) a Typora killer - Benjamin D. Lee
- 10 Problems with Obsidian - Medium
- Obsidian Regex Find and Replace - Forum
- Obsidian Regex Search in Files - Forum
- A Better Find & Replace UI - Mark Alldritt
- Find & Replace UX Ideas - VS Code GitHub
- CotEditor - Lightweight Plain-Text Editor for macOS
- CotEditor - Hacker News Discussion
- The State of Mac Code Editors - Collin Donnell
- IDE vs Text Editor - GeeksforGeeks
- Text Editor vs IDE - UltraEdit
- Midnight Commander mcedit Manual
- Total Commander Feature List
- Path Finder - XDA Developers
- Large File Text Editors - codegenes.net
- The Memory Bottleneck - Medium
- JSON vs YAML vs TOML - DEV Community
- EditorConfig
- Sublime Text Search and Replace - Docs
- Multiple Cursors Feature - AlternativeTo
- Simultaneous Editing - Wikipedia